

题目链接

onlinejudge

题意

枯燥的上下班路上我会打开随声听听歌，歌曲一共n首（ $1 \leq n \leq 75$ ），我使用随机模式听歌：每次启动时，随机选择一首歌曲播放，此曲播放完就会剔除它，下次继续随机选择其他歌曲播放，直到所有歌曲都播放过或者我按停止键，下次启动随机播放时，那些已播放的歌曲被选中的权重是2而从未播放过的歌曲被选中的权重是1。假设我上班路上不稳定的交通状况使我能等概率地听完p1、p1+1、p1+2、...、q1首歌，下班的交通状况使我能等概率地听完q1、q1+1、q1+2、...、q2首歌（ $0 \leq p1, q1, p2, q2 \leq 30, p1 \leq q1, p2 \leq q2$ ），问平均多少天我能听完所有歌曲。

分析

概率题，首先能想到用马尔可夫做状态递推：用f[i]表示在已经听过i首歌的条件下听完所有歌曲需要的天数，则f[n]=0。然后根据上班等概率听[p1,q1]首歌下班等概率听[p2,q2]首歌以及权重变化依次计算f[n-1]、f[n-2]、...、f[0]的递推式求解。

本题的数据规模庞大，需要借助dp做预处理，计算f[i]向f[i]、f[i+1]、f[i+2]、...、f[n]的转化概率：用d[n][i][p][q][a]表示总共n首歌已听i首，每次随机播放能等概率听完[p,q]首歌，停止播放时能额外新听a首歌的概率。

更多细节，参见AC代码。

AC代码

```
#include <iostream>
#include <iomanip>
using namespace std;

#define M 30
#define N 76
double d[N][N-1][M+1][M+1][M+1], e[N], f[N]; int n;

void solve() {
    int n, p1, q1, p2, q2; cin >> n >> p1 >> q1 >> p2 >> q2;
    if (q1==0 && q2==0) {
        cout << "IMPOSSIBLE" << endl; return;
    }
    if (p1>n || p2>n) {
        cout << 1. << endl; return;
    }
    f[n] = 0.; for (int i=0; i<n; ++i) f[i] = 1.;
    for (int i=n-1; i>=0; --i) {
        int a = min(n-i, q1), b = min(n-i, q1+q2);
        for (int j=0; j<=b; ++j) e[j] = 0.;
        for (int j=0; j<=a; ++j) {
            double &r = d[n][i][p1][q1][j];
            if (j+i == n) e[j] += r;
            else for (int k=0, s=i+j, m=min(n-s,q2); k<=m; ++k) e[j+k] += r*d[n][s][p2][q2][k];
        }
        for (int j=1; j<=b; ++j) f[i] += e[j]*f[i+j];
        f[i] /= 1. - e[0];
    }
    cout << f[0] << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    cout << fixed << setprecision(6);
    for (int i=1; i<N; ++i) for (int j=0; j<i; ++j) {
        d[i][j][0][0][0] = 1.; for (int k=1; k<=M; ++k) d[i][j][0][0][k] = 0.;
        for (int k=0; k<M; ++k) {
            for (int s=0; s<=k; ++s) for (int p=0; p<=M; ++p) d[i][j][s][k+1][p] = 0.;
            if (k+1 <= j) d[i][j][k+1][k+1][0] = d[i][j][k][k][0] * 2*(j-k) / (i+j-2*k);
            if (k+1 >= i) d[i][j][k+1][k+1][i-j] = 1.;
            else for (int p=max(k-j, 0), q=min(i-j,k+1); p<q; ++p) {
                double &r = d[i][j][k+1][k+1][p+1] = d[i][j][k][k][p] * (i-j-p) / (i+j+p-2*k);
                if (p+1 <= i-j && k-p-1 < j) r += d[i][j][k][k][p+1] * 2*(j+p+1-k) / (i+j+p+1-2*k);
            }
        }
        for (int k=1; k<=M; ++k) for (int s=k-1; s>=0; --s) for (int p=0, q=min(i-j, k); p<=q; ++p)
            d[i][j][s][k][p] = (d[i][j][s][s][p] + d[i][j][s+1][k][p]*(k-s))/(k-s+1);
    }
    int t; cin >> t;
    for (int i=1; i<=t; ++i) cout << "Case " << i << ": ", solve();
    return 0;
}
```